

Chapter 9. Deployment: Launching Your AI Application into Production

So far, we've explored the key concepts, ideas, and tools to help you build the core functionality of your AI application. You've learned how to utilize LangChain and LangGraph to generate LLM outputs, index and retrieve data, and enable memory and agency.

But your application is limited to your local environment, so external users can't access its features yet.

In this chapter, you'll learn the best practices for deploying your AI application into production. We'll also explore various tools to debug, collaborate, test, and monitor your LLM applications.

Let's get started.

Prerequisites

In order to effectively deploy your AI application, you need to utilize various services to host your application, store and retrieve data, and monitor your application. In the deployment example in this chapter, we will incorporate the following services:

Vector store

Supabase

Monitoring and debugging

LangSmith

Backend API

LangGraph Platform

We will dive deeper into each of these components and services and see how to adapt them for your use case. But first, let's install necessary dependencies and set up the environment variables.

If you'd like to follow the example, fork this [LangChain template](#) to your GitHub account. This repository contains the full logic of a retrieval

agent-based AI application.

Install Dependencies

First, follow the instructions in the [README.md file](#) to install the project dependencies.

If you're not using the template, you can install the dependencies individually from the respective *pyproject.toml* or *package.json* files.

Second, create a *.env* file and store the following variables:

```
OPENAI_API_KEY=  
SUPABASE_URL=  
SUPABASE_SERVICE_ROLE_KEY=  
  
# for tracing  
LANGCHAIN_TRACING_V2=true  
LANGCHAIN_ENDPOINT="https://api.smith.langchain.com"  
LANGCHAIN_API_KEY=
```

Next, we'll walk through the process of retrieving the values for each of these variables.

Large Language Model

The LLM is responsible for generating the output based on a given query. LangChain provides access to popular LLM providers, including OpenAI, Anthropic, Google, and Cohere.

In this deployment example, we'll utilize OpenAI by retrieving the [API keys](#), as shown in [Figure 9-1](#). Once you've retrieved your API keys, input the value as `OPENAI_API_KEY` in your *.env* file.

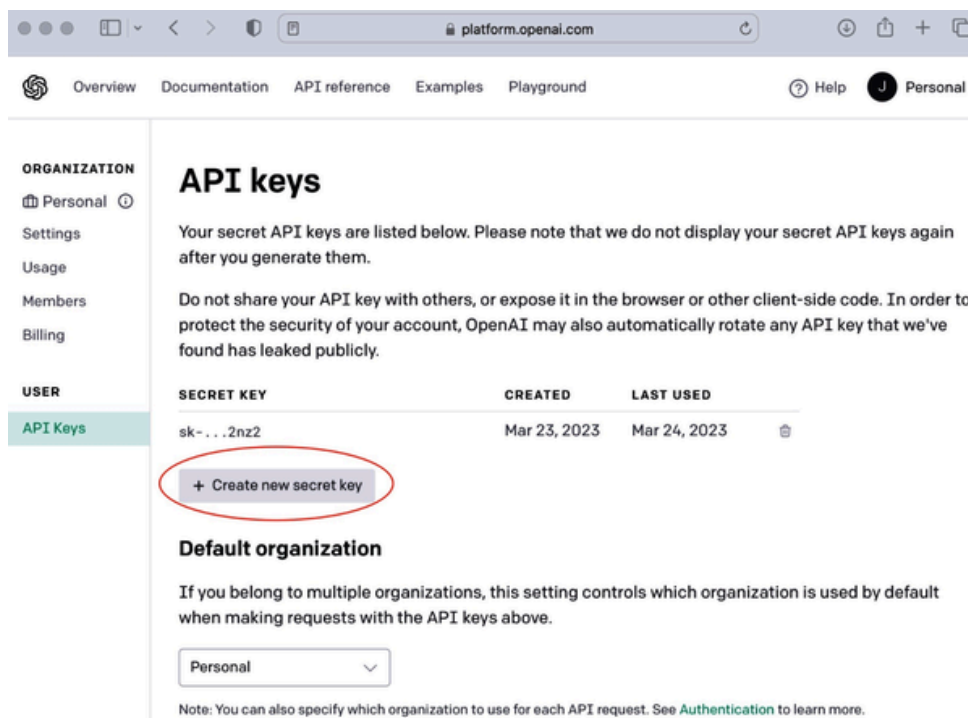


Figure 9-1. OpenAI API keys dashboard

Vector Store

As discussed in previous chapters, a vector store is a special database responsible for storing and managing vector representations of your data—in other words, embeddings. A vector store enables similarity search and context retrieval to help the LLM generate accurate answers based on the user’s query.

For our deployment, we’ll use Supabase—a PostgreSQL database—as the vector store. Supabase utilizes the `pgvector` extension to store embeddings and query vectors for similarity search.

If you haven’t yet done it, create a [Supabase account](#). Once you’ve created an account, click “New project” on the dashboard page. Follow the steps and save the database password after creating it, as shown in [Figure 9-2](#).

Create a new project
Your project will have its own dedicated instance and full Postgres database.
An API will be set up so you can easily interact with your new database.

Organization

Project name

Database Password
This is the password to your postgres database, so it must be strong and hard to guess. [Generate a password](#)

Region
Select the region closest to your users for the best performance.

SECURITY OPTIONS >

You can rename your project later

Figure 9-2. Supabase project creation dashboard

Once your Supabase project is created, navigate to the Project Settings tab and select API under Configuration. Under this new tab, you will see Project URL and Project API keys.

In your `.env` file, copy and paste the Project URL as the value to `SUPABASE_URL` and the `service_role` secret API key as the value to `SUPABASE_SERVICE_ROLE_KEY`.

Navigate to the SQL editor in the Supabase menu and run the following SQL scripts. First, let's enable `pgvector`:

```
## Enable the pgvector extension to work with embedding vectors
create extension vector;
```

Now create a table called `documents` to store vectors of your data:

```
## Create a table to store your documents

create table documents (
  id bigserial primary key,
  content text, -- corresponds to Document.pageContent
  metadata jsonb, -- corresponds to Document.metadata
  embedding vector(1536) -- 1536 works for OpenAI embeddings, change if nee
);
```

You should now see the `documents` table in the Supabase database.

Now you can create a script to generate the embeddings of your data, store them, and query from the database. Open the Supabase SQL editor again and run the following script:

```
## Create a function to search for documents
create function match_documents (
  query_embedding vector(1536),
  match_count int DEFAULT null,
  filter jsonb DEFAULT '{}'
) returns table (
  id bigint,
  content text,
  metadata jsonb,
  embedding jsonb,
  similarity float
)
language plpgsql
as $$
#variable_conflict use_column
begin
  return query
  select
    id,
    content,
    metadata,
    (embedding::text)::jsonb as embedding,
    1 - (documents.embedding <=> query_embedding) as similarity
  from documents
  where metadata @> filter
  order by documents.embedding <=> query_embedding
  limit match_count;
end;
$$;
```

The `match_documents` database function takes a `query_embedding` vector and compares it to embeddings in the `documents` table using cosine similarity. It calculates a similarity score for each document (`1 - (documents.embedding <=> query_embedding)`), then returns the most similar matches. The results are:

1. Filtered first by the metadata criteria specified in the filter argument (using JSON containment `@>`).
2. Ordered by similarity score (highest first).
3. Limited to the number of matches specified in `match_count`.

Once the vector similarity function is generated, you can use Supabase as a vector store by importing the class and providing the necessary parameters. Here's an example of how it works:

Python

```
import os

from langchain_community.vectorstores import SupabaseVectorStore
from langchain_openai import OpenAIEmbeddings
from supabase.client import Client, create_client

supabase_url = os.environ.get("SUPABASE_URL")
supabase_key = os.environ.get("SUPABASE_SERVICE_ROLE_KEY")
supabase: Client = create_client(supabase_url, supabase_key)

embeddings = OpenAIEmbeddings()

## Assuming you've already generated embeddings of your data

vector_store = SupabaseVectorStore(
    embedding=embeddings,
    client=supabase,
    table_name="documents",
    query_name="match_documents",
)

## Test that similarity search is working

query = "What is this document about?"
matched_docs = vector_store.similarity_search(query)

print(matched_docs[0].page_content)
```

JavaScript

```
import {
  SupabaseVectorStore
} from "@langchain/community/vectorstores/supabase";
import { OpenAIEmbeddings } from "@langchain/openai";

import { createClient } from "@supabase/supabase-js";

const embeddings = new OpenAIEmbeddings();

const supabaseClient = createClient(
  process.env.SUPABASE_URL,
  process.env.SUPABASE_SERVICE_ROLE_KEY
);

const vectorStore = new SupabaseVectorStore(embeddings, {
  client: supabaseClient,
  tableName: "documents",
  queryName: "match_documents",
```

```

});

// Example documents structure of your data

const document1: Document = {
  pageContent: "The powerhouse of the cell is the mitochondria",
  metadata: { source: "https://example.com" },
};

const document2: Document = {
  pageContent: "Buildings are made out of brick",
  metadata: { source: "https://example.com" },
};

const documents = [document1, document2]

//Embed and store the data in the database

await vectorStore.addDocuments(documents, { ids: ["1", "2"] });

// Query the Vector Store

const filter = { source: "https://example.com" };

const similaritySearchResults = await vectorStore.similaritySearch(
  "biology",
  2,
  filter
);

for (const doc of similaritySearchResults) {
  console.log(`* ${doc.pageContent} [${JSON.stringify(doc.metadata, null)}]
}

```

The output:

```

The powerhouse of the cell is the mitochondria [{"source":"https://example

```

You can review the full logic of the Supabase vector store implementation in the Github LangChain template mentioned previously.

Backend API

As discussed in previous chapters, LangGraph is a low-level open source framework used to build complex agentic systems powered by LLMs. LangGraph enables fine-grained control over the flow and state of your application, built-in persistence, and advanced human-in-the-loop and memory features. [Figure 9-3](#) illustrates LangGraph's control flow.

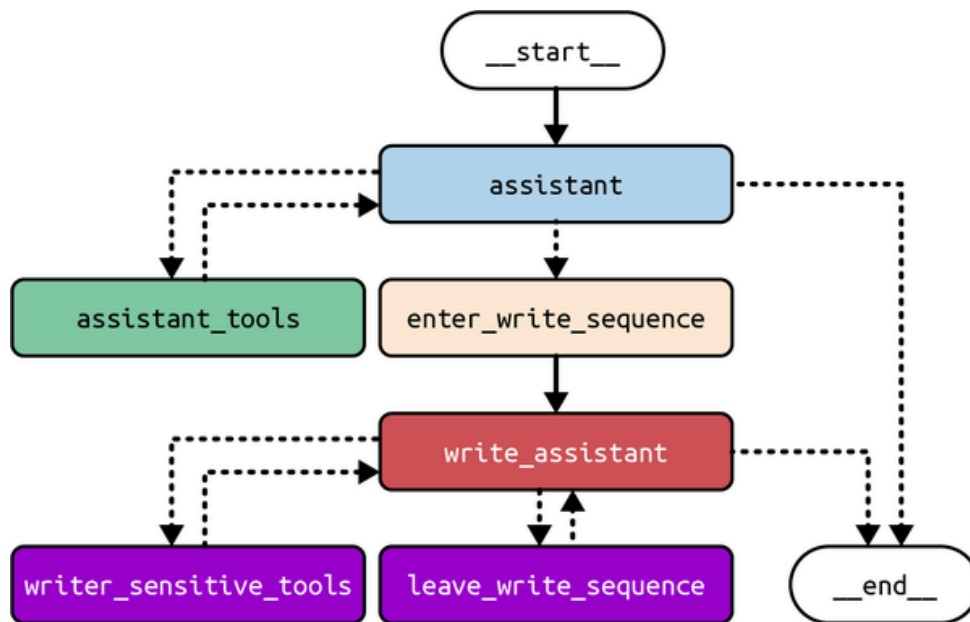


Figure 9-3. Example of LangGraph API control flow

To deploy an AI application that utilizes LangGraph, we will use LangGraph Platform. LangGraph Platform is a managed service for deploying and hosting LangGraph agents at scale.

As your agentic use case gains traction, uneven task distribution among agents can overload the system, leading to downtime. LangGraph Platform manages horizontally scaling task queues, servers, and a robust Postgres checkpointer to handle many concurrent users and efficiently store large states and threads. This ensures fault-tolerant scalability.

LangGraph Platform is designed to support real-world interaction patterns. In addition to streaming and human-in-the-loop features, LangGraph Platform enables the following:

- Double texting to handle new user inputs on ongoing graph threads
- Asynchronous background jobs for long-running tasks
- Cron jobs for running common tasks on a schedule

LangGraph Platform also provides an integrated solution for collaborating on, deploying, and monitoring agentic AI applications. It includes [LangGraph Studio](#)—a visual playground for debugging, editing, and testing agents. LangGraph Studio also enables you to share your LangGraph agent with team members for collaborative feedback and rapid iteration, as [Figure 9-4](#) shows.

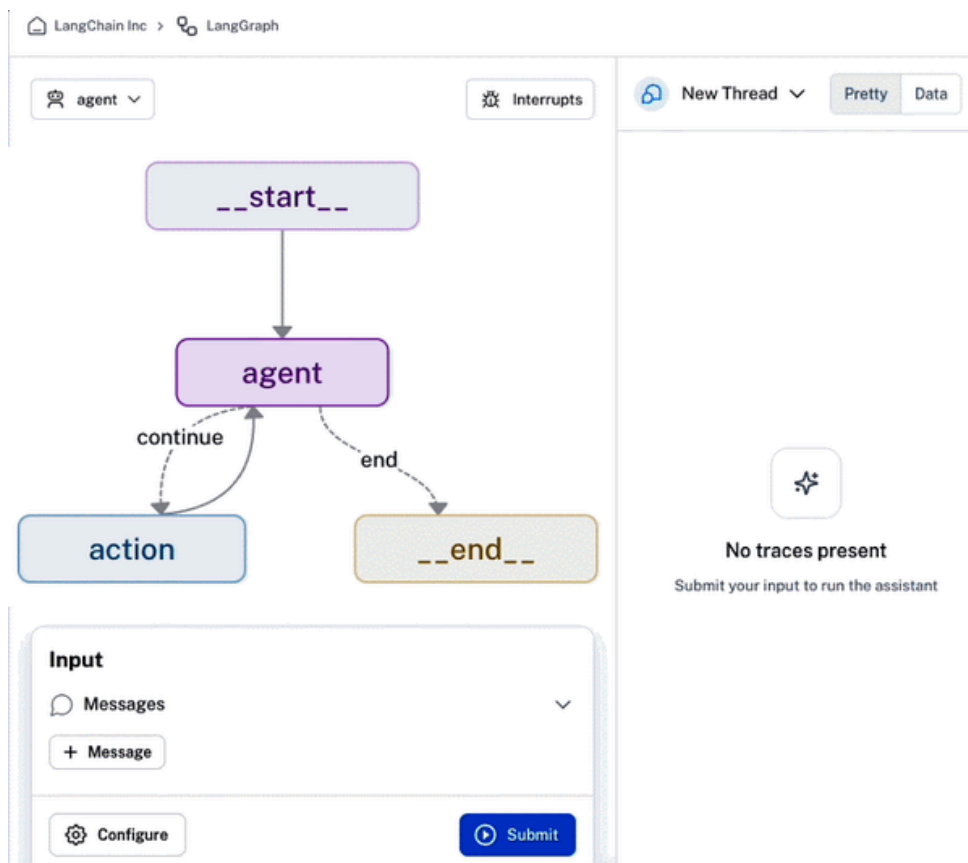


Figure 9-4. Snapshot of LangGraph Studio UI

Additionally, LangGraph Platform simplifies agentic deployment by enabling one-click submissions.

Create a LangSmith Account

LangSmith is an all-in-one developer platform that enables you to debug, collaborate, test, and monitor your LLM applications. LangGraph Platform is seamlessly integrated with LangSmith and is accessible from within the LangSmith UI.

To deploy your application on LangGraph Platform, you need to create a [LangSmith account](#). Once you're logged in to the dashboard, navigate to the Settings page, then scroll to the API Keys section and click Create API Key. You should see a UI similar to [Figure 9-5](#).

Figure 9-5. Create LangSmith API Key UI

Copy the API Key value as your `LANGCHAIN_API_KEY` in your `.env` file.

Navigate to “Usage and billing” and set up your billing details. Then click the “Plans and Billings” tab and the “Upgrade to Plus” button to get instructions on transitioning to a LangSmith Plus plan, which will enable LangGraph Platform usage. If you’d prefer to use a free self-hosted deployment, you can follow the [instructions here](#). Please note that this option requires management of the infrastructure, including setting up and maintaining required databases and Redis instances.

Understanding the LangGraph Platform API

Before deploying your AI application on LangGraph Platform, it’s important to understand how each component of the LangGraph API works. These components can generally be split into data models and features.

Data Models

The LangGraph Platform API consists of a few core data models:

- Assistants
- Threads
- Runs
- Cron jobs

Assistants

An *assistant* is a configured instance of a `CompiledGraph`. It abstracts the cognitive architecture of the graph and contains instance-specific configuration and metadata. Multiple assistants can reference the same graph but can contain different configuration and metadata—which may differentiate the behavior of the assistants. An assistant (that is, the graph) is invoked as part of a run.

The LangGraph Platform API provides several endpoints for creating and managing assistants.

Threads

A *thread* contains the accumulated state of a group of runs. If a run is executed on a thread, then the state of the underlying graph of the assistant will be persisted to the thread. A thread's current and historical state can be retrieved. To persist state, a thread must be created prior to executing a run. The state of a thread at a particular point in time is called a *checkpoint*.

The LangGraph Platform API provides several endpoints for creating and managing threads and thread state.

Runs

A *run* is an invocation of an assistant. Each run may have its own input, configuration, and metadata—which may affect the execution and output of the underlying graph. A run can optionally be executed on a thread.

The LangGraph Platform API provides several endpoints for creating and managing runs.

Cron jobs

LangGraph Platform supports *cron jobs*, which enable graphs to be run on a user-defined schedule. The user specifies a schedule, an assistant, and an input. Then LangGraph Platform creates a new thread with the specified assistant and sends the specified input to that thread.

Features

The LangGraph Platform API also offers several features to support complex agent architectures, including the following:

- Streaming

- Human-in-the-loop
- Double texting
- Stateless runs
- Webhooks

Streaming

Streaming is critical for ensuring that LLM applications feel responsive to end users. When creating a streaming run, the streaming mode determines what data is streamed back to the API client. The LangGraph Platform API supports five streaming modes:

Values

Stream the full state of the graph after each super-step is executed.

Messages

Stream complete messages (at the end of node execution) as well as tokens for any messages generated inside a node. This mode is primarily meant for powering chat applications. This is only an option if your graph contains a `messages` key.

Updates

Stream updates to the state of the graph after each node is executed.

Events

Stream all events (including the state of the graph) that occur during graph execution. This can be used to do token-by-token streaming for LLMs.

Debug

Stream debug events throughout graph execution.

Human-in-the-loop

If left to run autonomously, a complex agent can take unintended actions, leading to catastrophic application outcomes. To prevent this, human intervention is recommended, especially at checkpoints where application logic involves invoking certain tools or accessing specific documents. LangGraph Platform enables you to insert this human-in-the-loop behavior to ensure your graph doesn't have undesired outcomes.

Double texting

Graph execution may take longer than expected, and often users may send one message and then, before the graph has finished running, send a second message. This is known as *double texting*. For example, a user might notice a typo in their original request and edit the prompt and re-send it. In such scenarios, it's important to prevent your graphs from behaving in unexpected ways and ensure a smooth user experience.

LangGraph Platform provides four different solutions to handle double texting:

Reject

This rejects any follow-up runs and does not allow double texting.

Enqueue

This option continues the first run until it completes the whole run, then sends the new input as a separate run.

Interrupt

This option interrupts the current execution but saves all the work done up until that point. It then inserts the user input and continues from there. If you enable this option, your graph should be able to handle weird edge cases that may arise.

Rollback

This option rolls back all work done up until that point. It then sends the user input in—as if it just followed the original run input.

Stateless runs

All runs use the built-in checkpointer to store checkpoints for runs. However, it can often be useful to just kick off a run without worrying about explicitly creating a thread and keeping those checkpointers around. *Stateless* runs allow you to do this by exposing an endpoint that does these things:

- Takes in user input
- Creates a thread
- Runs the agent, but skips all checkpointing steps
- Cleans up the thread afterwards

Stateless runs are retried while keeping memory intact. However, in the case of stateless background runs, if the task worker dies halfway, the entire run will be retried from scratch.

Webhooks

LangGraph Platform also supports completion *webhooks*. A webhook URL is provided, which notifies your application whenever a run completes.

Deploying Your AI Application on LangGraph Platform

At this point, you have created accounts for the recommended services, filled in your `.env` file with values of all necessary environment variables, and completed the core logic for your AI application. Next, we will take the necessary steps to effectively deploy your application.

Create a LangGraph API Config

Prior to deployment, you need to configure your application with a [LangGraph API configuration file called `langgraph.json`](#). Here's an example of what the file looks like in a Python repository:

Python

```
{
  "dependencies": ["/my_agent"],
  "graphs": {
    "agent": "/my_agent/agent.py:graph"
  },
  "env": ".env"
}
```

And here's an example repository structure:

```
my-app/
├── my_agent # all project code lies within here
│   ├── utils # utilities for your graph
│   │   ├── __init__.py
│   │   ├── tools.py # tools for your graph
│   │   ├── nodes.py # node functions for you graph
│   │   └── state.py # state definition of your graph
│   ├── requirements.txt # package dependencies
│   ├── __init__.py
│   └── agent.py # code for constructing your graph
├── .env # environment variables
└── langgraph.json # configuration file for LangGraph
```

Note that the `langgraph.json` file is placed on the same level or higher than the files that contain compiled graphs and associated dependencies.

In addition, the dependencies are specified in a *requirements.txt* file. But they can also be specified in *pyproject.toml*, *setup.py*, or *package.json* files.

Here's what each of the properties mean:

Dependencies

Array of dependencies for LangGraph Platform API server

Graphs

Mapping from graph ID to path where the compiled graph or a function that makes a graph is defined

Env

Path to your *.env* file or a mapping from environment variable to its value (you can learn more about configurations for the `langgraph.json` file [here](#))

Test Your LangGraph App Locally

Testing your application locally ensures that there are no errors or dependency conflicts prior to deployment. To do this, we will utilize the LangGraph CLI, which includes commands to run a local development server with hot reloading and debugging capabilities.

For Python, install the Python `langgraph-cli` package (note: this requires Python 3.11 or higher):

```
pip install -U "langgraph-cli[inmem]"
```

Or for JavaScript, install the package as follows:

```
npm i @langchain/langgraph-cli
```

Once the CLI is installed, run the following command to start the API:

```
langgraph dev
```

This will start up the LangGraph API server locally. If this runs successfully, you should see something like this:

```
Ready!  
API: http://localhost:2024  
Docs: http://localhost:2024/docs
```

The LangGraph Platform API reference is available with each deployment at the `/docs` URL path (<http://localhost:2024/docs>).

The easiest way to interact with your local API server is to use the auto-launched LangGraph Studio UI. Alternatively, you can interact with the local API server using cURL, as seen in this example:

```
curl --request POST \
  --url http://localhost:8123/runs/stream \
  --header 'Content-Type: application/json' \
  --data '{
    "assistant_id": "agent",
    "input": {
      "messages": [
        {
          "role": "user",
          "content": "How are you?"
        }
      ]
    },
    "metadata": {},
    "config": {
      "configurable": {}
    },
    "multitask_strategy": "reject",
    "stream_mode": [
      "values"
    ]
  }'
```

If you receive a valid response, your application is functioning well. Next, we can interact with the server using the LangGraph SDK.

Here's an example both initializing the SDK client and invoking the graph:

Python

```
from langgraph_sdk import get_client

# only pass the url argument to get_client() if you changed the default port
# when calling langgraph up
client = get_client()
# Using the graph deployed with the name "agent"
assistant_id = "agent"
thread = await client.threads.create()

input = {"messages": [{"role": "user", "content": "what's the weather in s"}]
async for chunk in client.runs.stream(
```

```

        thread["thread_id"],
        assistant_id,
        input=input,
        stream_mode="updates",
    ):
        print(f"Receiving new event of type: {chunk.event}...")
        print(chunk.data)
        print("\n\n")

```

JavaScript

```

import { Client } from "@langchain/langgraph-sdk";

// only set the apiUrl if you changed the default port when calling langgraph
const client = new Client();
// Using the graph deployed with the name "agent"
const assistantId = "agent";
const thread = await client.threads.create();

const input = {
  messages: [{ "role": "user", "content": "what's the weather in sf" }]
}

const streamResponse = client.runs.stream(
  thread["thread_id"],
  assistantId,
  {
    input: input,
    streamMode: "updates",
  }
);
for await (const chunk of streamResponse) {
  console.log(`Receiving new event of type: ${chunk.event}...`);
  console.log(chunk.data);
  console.log("\n\n");
}

```

If your LangGraph application is working correctly, you should see your graph output displayed in the console.

Deploy from the LangSmith UI

At this point, you should have completed all prerequisite steps and your LangGraph API should be working locally. Your next step is to navigate to your LangSmith dashboard panel and click the Deployments tab. You should see a UI similar to [Figure 9-6](#).

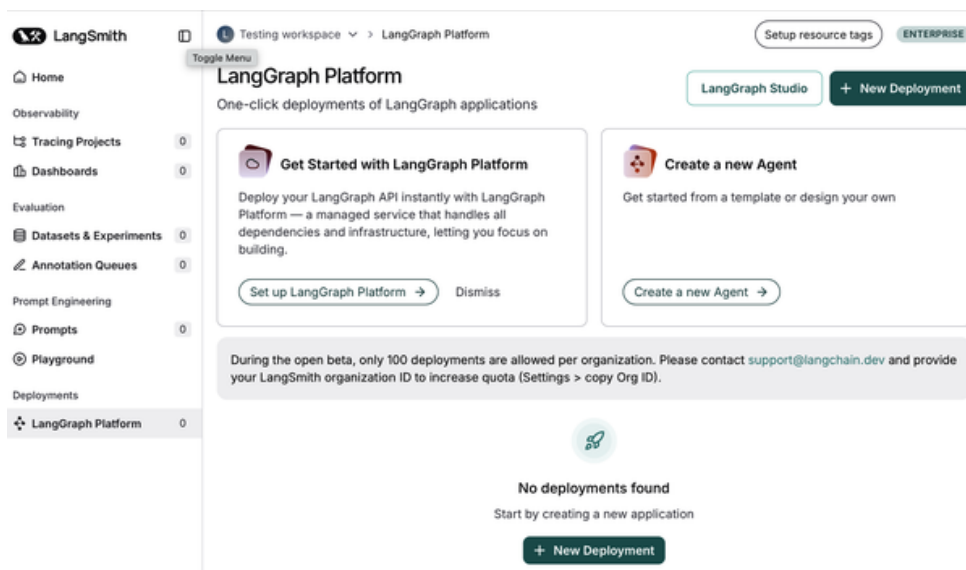


Figure 9-6. LangGraph Platform deployment UI page

Next, click the New Deployment button in the top right corner of the page.

NOTE

If you don't see a page with the New Deployment button, it's likely that you haven't yet upgraded to a LangSmith Plus plan according to the instructions in the "Usage and billing" setting.

You should now see a page of three form fields to complete.

Deployment details

1. Select "Import with GitHub" and follow the GitHub OAuth workflow to install and authorize LangChain's hosted-langserve GitHub app to access the selected repositories. After installation is complete, return to the Create New Deployment panel and select the GitHub repository to deploy from the drop-down menu.
2. Specify a name for the deployment and the full path to the LangGraph API config file, including the filename. For example, if the file *lang-graph.json* is in the root of the repository, simply specify *langgraph.json*.
3. Specify the desired `git` reference (branch name) of your repository to deploy.

Development type

Select Production from the dropdown. This will enable a production deployment that can serve up to 500 requests/second and is provisioned with highly available storage and automatic backups.

Environment variables

Provide the properties and values in your `.env` here. For sensitive values, like your `OPENAI_API_KEY`, make sure to tick the Secret box before inputting the value.

Once you've completed the fields, click the button to submit the deployment and wait for a few seconds for the build to complete. You should see a new revision associated with the deployment.

Since LangGraph Platform is integrated within LangSmith, you can gain deeper visibility into your app and track and monitor usage, errors, performance, and costs in production too. [Figure 9-7](#) shows a visual Trace Count summary chart showing successful, pending, and error traces over a given time period. You can also view all monitoring info for your server by clicking the “All charts” button.

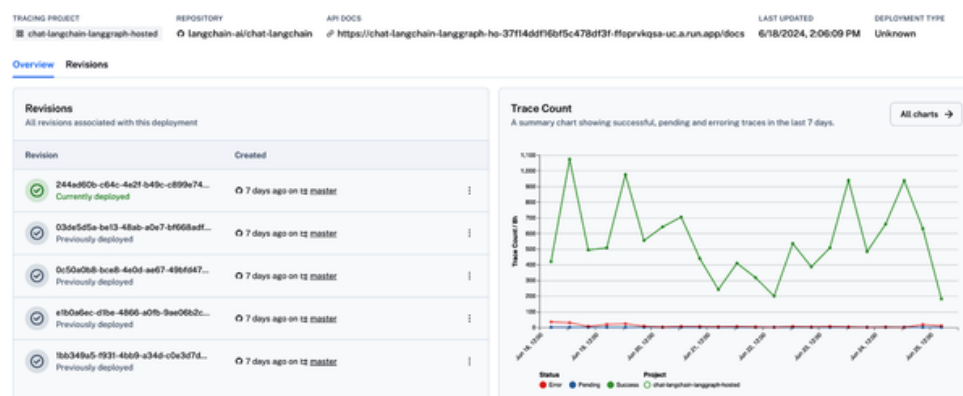


Figure 9-7. Deployment revisions and trace count on dashboard

To view the build and deployment logs, select the desired revision from the Revisions tab, then choose the Deploy tab to view the full deployment logs history. You can also adjust the date and time range.

To create a new deployment, click the New Revision button in the navigation bar. Fill out the necessary fields, including the LangGraph API config file path, git reference, and environment variables, as done previously.

Finally, you can access the API documentation by clicking the API docs link, which should display a similar page to the UI shown in [Figure 9-8](#).

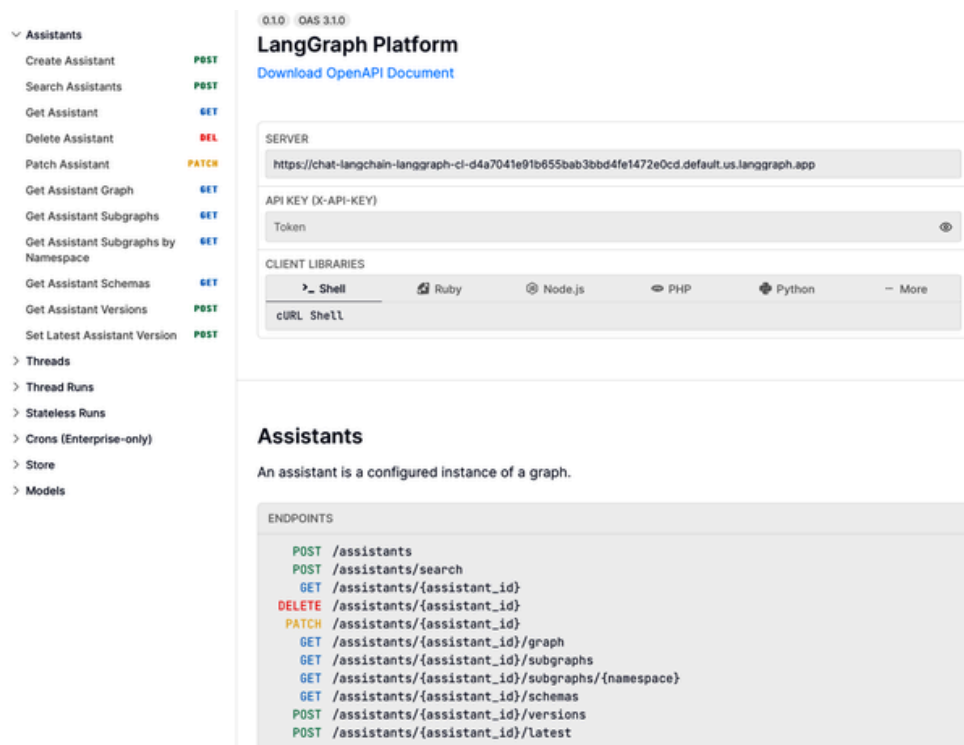


Figure 9-8. LangGraph API documentation

Launch LangGraph Studio

LangGraph Studio provides a specialized agent IDE for visualizing, interacting with, and debugging complex agentic applications. It enables developers to modify an agent result (or the logic underlying a specific node) halfway through the agent’s trajectory. This creates an iterative process by letting you interact with and manipulate the state at that point in time.

Once you’ve deployed your AI application, click the LangGraph Studio button at the top righthand corner of the deployment dashboard, as you can see in [Figure 9-9](#).



Figure 9-9. LangGraph deployment UI

After clicking the button, you should see the LangGraph Studio UI (for example, see [Figure 9-10](#)).

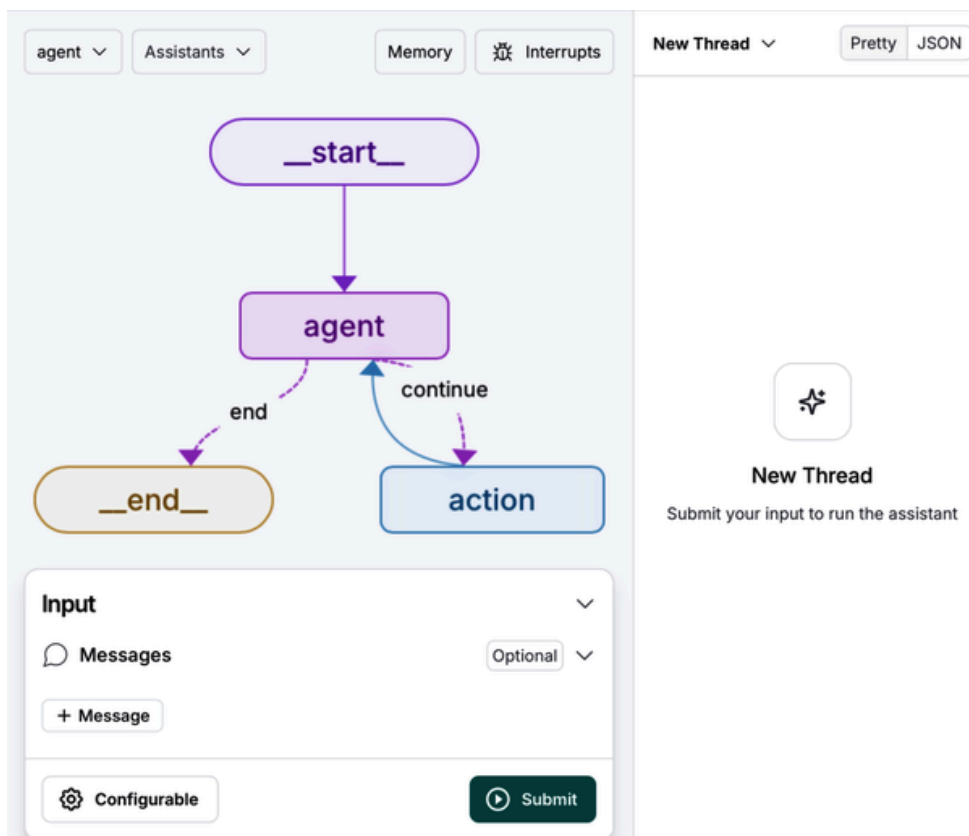


Figure 9-10. LangGraph Studio UI

To invoke a graph and start a new run, follow these steps:

1. Select a graph from the drop-down menu in the top left corner of the lefthand pane. The graph in [Figure 9-10](#) is called *agent*.
2. In the Input section, click the “+ Message” icon and input a *human* message, but the input will vary depending on your application state definitions.
3. Click Submit to invoke the selected graph.
4. View the output of the invocation in the right-hand pane.

The output of your invoked graph should look like [Figure 9-11](#).

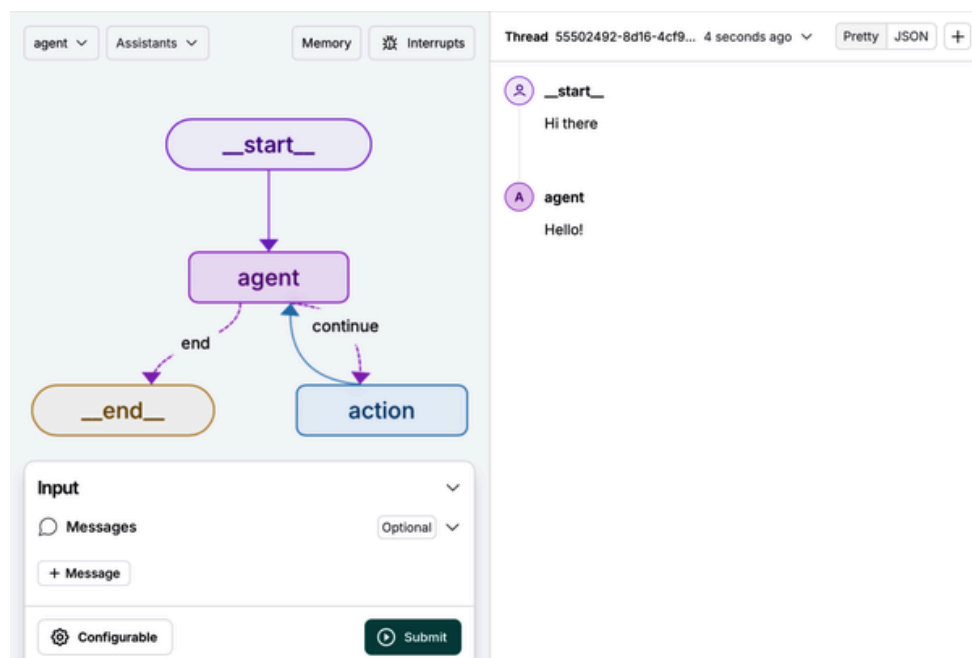


Figure 9-11. LangGraph Studio invocation output

In addition to invocation, LangGraph Studio enables you to change run configurations, create and edit threads, interrupt your graphs, edit graph code, and enable human-in-the-loop intervention. You can read the [full guide](#) to learn more.

NOTE

LangGraph Studio is also available as a desktop application (for Apple silicon), which enables you to test your AI application locally.

If you've followed the installation guide in the GitHub template and successfully deployed your AI application, it's now live for production use. But before you share to external users or use the backend API in existing applications, it's important to be aware of key security considerations.

Security

Although AI applications are powerful, they are vulnerable to several security risks that may lead to data corruption or loss, unauthorized access to confidential information, and compromised performance. These risks may carry adverse legal, reputational, and financial consequences.

To mitigate these risks, it's recommended to follow general application security best practices, including the following:

Limit permissions

Scope permissions specific to the application's need. Granting broad or excessive permissions can introduce significant security vulnerabilities. To avoid such vulnerabilities, consider using read-only credentials, disallowing access to sensitive resources, and using sandboxing techniques (such as running inside a container).

Anticipate potential misuse

Always assume that any system access or credentials may be used in any way allowed by the permissions they are assigned. For example, if a pair of database credentials allows deleting data, it's safest to assume that any LLM able to use those credentials may in fact delete data.

Defense in depth

It's often best to combine multiple layered security approaches rather than rely on any single layer of defense to ensure security. For example, use both read-only permissions and sandboxing to en-

sure that LLMs are only able to access data that is explicitly meant for them to use.

Here are three example scenarios implementing these mitigation strategies:

File access

A user may ask an agent with access to the file system to delete files that should not be deleted or read the content of files that contain sensitive information. To mitigate this risk, limit the agent to only use a specific directory and only allow it to read or write files that are safe to read or write. Consider further sandboxing the agent by running it in a container.

API access

A user may ask an agent with write access to an external API to write malicious data to the API or delete data from that API. To mitigate, give the agent read-only API keys, or limit it to only use endpoints that are already resistant to such misuse.

Database access

A user may ask an agent with access to a database to drop a table or mutate the schema. To mitigate, scope the credentials to only the tables that the agent needs to access and consider issuing read-only credentials.

In addition to the preceding security measures, you can take further steps to mitigate abuse of your AI application. Due to the dependency of external LLM API providers (such as OpenAI), there is a direct cost associated with running your application. To prevent abuse of your API and exponential costs, you can implement the following:

Account creation verification

This typically includes a form of authentication login, such as email or phone number verification.

Rate limiting

Implement a rate-limiting mechanism in the middleware of the application to prevent users from making too many requests in a short period of time. This should check the number of requests a user has made in the last X minutes and “timeout” or “ban” the user if the abuse is severe.

Implement prompt injection guardrails

Prompt injection occurs when a malicious user injects a prompt in an attempt to trick the LLM to act in unintended ways. This usually includes extracting confidential data or generating unrelated outputs. To mitigate this, you should ensure the LLM has proper permission scoping and that the application's prompts are specific and strict to the desired outcomes.

Summary

Throughout this chapter, you've learned the best practices for deploying your AI application and enabling users to interact with it. We explored recommended services to handle various key components of the application in production, including the LLM, vector store, and backend API.

We also discussed using LangGraph Platform as a managed service for deploying and hosting LangGraph agents at scale—in conjunction with LangGraph Studio—to visualize, interact with, and debug your application.

Finally, we briefly explored various security best practices to mitigate data breach risks often associated with AI applications.

In [Chapter 10](#), you'll learn how to effectively evaluate, monitor, benchmark, and improve the performance of your AI application.